

The Collection: technical guide

The repository, the component contract, the services, the configuration for the bank's systems, deployment and operation.

For	Engineers building with or contributing to the collection; platform and integration engineers deploying it; operators
Date	20 September 2026
Status	Reflects the repository at the date above; the readiness page in the repository is the live record

1. What is in the repository

Path	What it is
components/	29 AI components in six categories under four groups (agents, python, typescript, skills). Each directory is self-contained: a manifest, a README, a walkthrough, a live example, tests
hub/	The employee AI hub front end (React 18, TypeScript, Vite). Pixel-identical to its design artboards; a mock API for development; configured at runtime by hub-api
services/hub-api/	The hub's API on the Python standard library: every path of the contract plus sign-offs, behind the bank's identity provider; serves the built hub
services/agent-runtime/	One agent of the collection over MCP and a run API, wired to the bank's systems by configuration
tools/	shelf.py (the manifest checker and exporter), publish_kb.py (pages into the knowledge base), new_component.py (the scaffold), kb-taxonomy.json
content/knowledgebase/	Pages the collection authors for the knowledge base product (practices, paved roads, the programme, onboarding)
exports/knowledgebase/, SHELF.md, hub/src/api/mock/collection.ts, serv ices/hub-api/data/collection.json	Generated by tools/shelf.py --write; never edited by hand
config/, CONFIGURATION.md	Every external system named once; what the bank supplies
deploy/, services/*/deploy/	Compose for the sandbox; Dockerfiles, fail-closed entrypoints, ECS task definitions, task roles
scripts/	verify.sh (every gate), bundle.sh (the offline release), smoke-container-tree.sh
PRODUCTION-READINESS.md, RUNBOOK.md, SECURITY.md, HANDOVER.md, CONTRIBUTING.md	The operating pages and the contract

2. Components: the contract

A component is one directory with a **component.json** manifest. The shelf tool validates every manifest, checks that vendored copies are byte-identical to their source, checks tags against the knowledge base's taxonomy, runs every test and live example, and regenerates the exports. CI fails on any drift.

2.1 Categories

Category	What it is	Who uses it	What it must contain
agent	An AI agent	Operators; engineers deploy one	A template (role, stages, tools by tier, a never list), the tools it may call, the harness it runs inside, declared in the manifest's agent block
harness	The loop an agent runs inside	Engineers building an agent	Fixed hooks, action tiers, budgets, kill switches, a chained record
tool	Code with one clear surface	Engineers; an agent calls one through its harness	The code and a test that proves it

Category	What it is	Who uses it	What it must contain
integration	A client for an external system	Engineers connecting a system	The client and an in-memory fake behind the same methods
pattern	A reference implementation of a practice	Engineers adopting the practice	The implementation and its practice page
skill	A procedure	Anyone, nothing to run	SKILL.md with the procedure and its checks, a template, a filled example

2.2 The manifest

Fields: name, version (semantic; sign-offs bind to it), category, agent (agents only), signoff (owner and ai_security, each null or by/date/version), used_in, language, summary (under 160 characters), status (ready, draft, deprecated), source (project, path, snapshot), owner (a handle), tags (from the taxonomy), spec (the platform design specification sections and PLT ids implemented in interim form, and the replacement test), requires, pairs_with, test, walkthrough, example (path and run), vendored (path and from).

2.3 Rules

- Python components use the standard library only unless requires says otherwise; a component imports nothing outside its directory; shared files are vendored and declared.
- Every component carries a version, two sign-offs, a walkthrough and a live example; the hub lists it as generally available only when both sign-offs name the current version.
- Fakes ship with the real thing: an integration has an in-memory fake behind the same methods.
- No secrets, no real ids; credentials are names looked up at call time.
- A change a consumer would notice bumps the version, which makes both sign-offs stale.

2.4 The six stages to the shelf

Stage	What proves it
1 scaffolded	The scaffold made the directory: version 0.1.0, sign-offs pending, a spec entry, a tag
2 built	README, walkthrough, example and tests filled and green; status ready
3 used once for real	used_in names a project (the owner records it on the sign-off form)
4 owner signed	At this version, after running the tests and the example
5 AI security signed	At this version, after reading the rules and the walkthrough and running the example
6 on the shelf	Both at the current version: generally available on the hub and in the knowledge base

2.5 Signing off

On the hub: Build, Sign-offs, or the card on the component's page. The server requires every attestation and the first real use; the owner signs by name (the manifest's owner is their handle) and AI security by the ai.security role. The manifest is the record and the commit is the signature:

```
python3 tools/shelf.py --apply-signoffs shelf-signoffs.json # re-runs the tests, writes signoff.<role>
python3 tools/shelf.py --write
git commit -am "Sign off <component> <version>"
```

2.6 Adding a component

```
python3 tools/new_component.py python my-tool --category tool --summary "What it does"
python3 tools/new_component.py agents my-agent --category agent --summary "..." #
TEMPLATE.md first, then one test per never line
python3 tools/shelf.py --write && python3 tools/shelf.py --test --only python && python3
tools/shelf.py --examples --only python
```

3. The harness and the agent

governed-action-loop is the harness: three fixed hooks on every call (before, after, record), action tiers R, W1, W2 and MONEY, a W1 call parked until the acting person confirms its exact hash, W2 under dual control, a taint ceiling that caps a tainted session at reads, kill switches, budgets, and a hash-chained record. It ships with a fake identity provider, a local signing key and a fake gateway, and with their production counterparts behind the same interfaces: **JwksIdP** (the bank's RS256 tokens through its JWKS, directory groups to roles) and **KmsKey** (catalog signing on an asymmetric KMS key through SigV4 from the task role).

incident-first-read-agent is the reference agent. Its TEMPLATE.md is loaded as data and the harness's signed catalog is built from its tools, so the agent can call exactly what the template lists. It reads a ticket and the last deploy, produces a cited first read through the **cited-llm-engine** (rules offline, a model online; fenced sources, cite-or-drop, a proposal refused on taint), and parks the W1 comment. One test exists per line of its never list.

4. MCP

- **mcp-tool-server** (harness): JSON-RPC over stdio or Streamable HTTP in front of the loop. tools/list is the signed catalog with annotations derived from it; every tools/call runs the hooks; W1 is an elicitation; taint, ladder, scope and kill switches are 403 with WWW-Authenticate insufficient_scope; RFC 9728 metadata; sampling disabled.
- **mcp-gateway-client** (integration): the harness's gateway for an external MCP server under a contract record: an allowlist pinned to description hashes, quarantine on any drift or on a description that scores as an injection, the credential a name.
- **shelf-mcp-server** (tool): read-only tools and every README, walkthrough and template as resources, so a coding assistant discovers the collection without cloning it. One line registers it with Claude Code.

5. The services

5.1 hub-api

Area	Behaviour
Identity	HUB_AUTH=oidc: RS256 through the JWKS (signature, exp, nbf, iss, aud), claims mapped by identity-map.json (group ids to roles, entitlements, teams, ladder). HUB_AUTH=mock: personas for development, refused in production
Catalog	The bank's listings from HUB_CONSUMERS_FILE plus every component from collection.json; access resolved per person from entitlements
Briefs, requests, workspace, preferences	The browser's rules mirrored server-side; etags with If-Match; the lead rule for write profiles; persisted per person

Area	Behaviour
Conversations	SSE view events from one of three assistants: fake, http (relay to the platform runtime, service credential by name), bedrock (sources from the knowledge base's search fenced by the guard, cite-or-drop, taint stops before the model)
The shelf	Sign-offs recorded against the version signed at; owner by name, AI security by role; every attestation required; the export the shelf tool applies
Runtime configuration	/config.js rendered from HUB_WEB_* so one built hub runs in every environment
Static, record, health	The hub's dist served with an SPA fallback; SQLite at HUB_DB (a file in staging and production); /api/health anonymous; logs ids only; request bodies capped; SIGTERM finishes in-flight requests

5.2 agent-runtime

Area	Behaviour
Surface	/mcp (the template's tools; W1 as elicitation), /runs (a first read: read, think, propose, park), /runs/{session}/confirm (the person confirms the exact hash; runs once), /health
Identity	AGENT_IDENTITY=oidc through the JWKS with groups to the operator and approver roles; fake in the sandbox
Signing	AGENT_SIGNING=kms; local in the sandbox
Think step	AGENT_ENGINE=bedrock through the VPC endpoint with the template's role as the system prompt; rules in the sandbox
Targets	AGENT_TARGETS names which of the template's targets are real: tickets is the Jira connector, deploys the Azure DevOps connector; the rest are fakes, refused in staging and production. The targets a template needs are read from the template
Record	SQLite on a volume; verify-record at every start; export-audit puts the chain and a latest.json pointer to S3 with SigV4

6. Configuring for the bank's systems

Every variable is in **config/collection.env.example** and explained in **CONFIGURATION.md**. A variable ending in **_NAME** is the name of a secret in the secrets provider (HUB_SECRETS=aws is Secrets Manager reached with the task role; file: a mounted file; env only for a developer's shell). Each service's settings fail closed: check-config lists every problem by variable name, never a value, and the entrypoint runs it before the process listens.

System	Variables	Needed from the bank
Identity provider	HUB_IDP_ISSUER, HUB_IDP_AUDIENCE, HUB_WEB_OIDC_AUTHORITY, HUB_WEB_OIDC_CLIENT_ID, AGENT_IDP_*	An app registration per deployable with the groups claim; the redirect URI
Groups to grants	HUB_IDENTITY_MAP, HUB_AI_SECURITY_GROUP, AGENT_OPERATOR_GROUP_ID, AGENT_APPROVER_GROUP_ID	Which groups grant which roles, entitlements, teams and ladders
Listings	HUB_CONSUMERS_FILE	The bank's own consumers and the registry of systems and tools
Record	HUB_DB, AGENT_DB	A persistent volume (EFS)

System	Variables	Needed from the bank
Model	*_BEDROCK_*	An inference profile approved by model risk; the VPC endpoint; bedrock:InvokeModel on the task role
Secrets	*_SECRETS, the *_NAME variables	Secrets under the service prefix; secretsmanager:GetSecretValue on the task role
Catalog signing	AGENT_KMS_KEY_ID	An asymmetric KMS key; kms:Sign and kms:GetPublicKey
Jira, Azure DevOps	AGENT_JIRA_*, AGENT_DEPLOYS_*	A service account with read on incident projects and comment; a PAT scoped to pipelines
Knowledge base	HUB_KB_URL, HUB_KB_SEARCH_URL	The console URL; its /search endpoint for citations
Audit export	AGENT_AUDIT_EXPORT	A bucket with KMS encryption and s3:PutObject on the task role

Sandbox allows mock identity, the fake assistant, the rules engine, local signing and fake targets. Staging and production refuse all of them, require https public URLs, a file-backed record, a non-environment secrets provider, KMS signing and, for the agent, a real system for every target its template names. The same image runs in both; only the environment changes.

7. Deployment

- 1 Build the images** from the repository root with the base image digest pinned to the bank's approved image: `docker build -f services/hub-api/deploy/Dockerfile` and `services/agent-runtime/deploy/Dockerfile`. The hub's dist is prebuilt (the release bundle ships it) and needs no rebuild for a new environment.
- 2 Mount the configuration:** `identity-map.json` and `consumers.json` on the hub's read-only config volume.
- 3 Create the secrets** under `hub/` and `agents/`, and the task roles from `services/*/deploy/iam-task-role-policy.json`.
- 4 Register the task definitions** from `services/*/deploy/ecs-task-definition.json` with the record volumes on EFS.
- 5 Verify:** `/api/health` and `/health` answer; a real bearer resolves to the expected principal; the record survives a task restart; the audit export lands in the bucket.

Locally, with fakes and no account: `docker compose -f deploy/compose.yaml up`. Without a container daemon: `scripts/smoke-container-tree.sh` assembles each Dockerfile's COPY set, runs the entrypoint, checks health and that production refuses the tree's fakes.

8. Verification, CI and the offline release

```
scripts/verify.sh python    # the shelf, the vendoring, every Python component and service,
                             the refusals, the container trees
scripts/verify.sh typescript
scripts/verify.sh hub       # typecheck, lint, 43 tests, build
scripts/bundle.sh           # release/collection-<sha>.tar.gz with the hub prebuilt and a
                             sha256 manifest
```

.github/workflows/ci.yml and ci/azure-pipelines.yml both call verify.sh, so the two runners cannot drift. The bundle, unpacked in a clean directory, verifies its manifest and passes the Python gates with no network and no Node. Services vendor component files through services/vendor.json; python3 services/vendor.py --check refuses drift.

9. Operating

RUNBOOK.md covers health, what a refused start looks like, rolling out a build, applying a sign-off, changing listings or grants, stopping the agent, rotating a credential, exporting the chain, and what to do when the record does not verify (restore, compare with the last export, open an incident). SECURITY.md maps each threat to its control and its test.

10. The knowledge base

Pages under content/knowledgebase/ and the generated component pages obey the product's checks (frontmatter, under 200 lines, organisation-neutral, links inside docs/, list items on one line). Deliver with python3 tools/publish_kb.py <checkout>, then the product's kb-librarian index --write and check. The publisher is idempotent.

11. Where the platform replaces the interim

Every component's manifest names the platform design specification sections and PLT ids it implements in interim form and a replacement test: the condition the platform must pass before the component is retired. The hub shows both on each listing. The MCP tool server is the specification's R1 road with the harness behind the transport; the gateway client is the contract registry's rules for external servers; the harness's stores and gateway are the control layer's when it arrives.